

OMERTA AI

Linux Mint — Full Build Specification

Phase 1 → Phase 15 • Engineering / Coding / Software / Firmware AI

Purpose: This is the master technical specification for building OMERTA AI on Linux Mint. It defines the architecture, features, tools, sandbox, learning system, agent roles, firmware/device-tree capabilities, build/debug pipeline, security model and release plan discussed for the project.

Design principle: direct answers, evidence-backed execution, explicit learning, reproducible engineering, sandbox-first execution, and no claims of success without verification.

1. System Requirements — Linux Mint

Component	Baseline	Preferred
OS	Linux Mint 22.x 64-bit	Current supported Linux Mint 22.x release
CPU	64-bit x86_64, 4 cores	8+ cores
RAM	8 GB	16–32 GB
Storage	50 GB free	150+ GB SSD
Swap	8 GB	16+ GB for large builds
Python	3.12+	3.12/3.13
Git	2.4+	Current Mint package
Container runtime	Docker or Podman	Docker/Podman + bubblewrap
GPU	Optional	AMD/NVIDIA GPU for local model workloads

For a low-RAM machine, OMERTA should default to cloud models and lightweight local indexing; large Android/kernel builds should run with controlled concurrency.

2. Complete Architecture

OMERTA AI ■■■ CLI / Web UI / API ■■■ Agent Core ■ ■■■ Planner ■ ■■■ Executor ■ ■■■ Context Manager ■ ■■■ Evidence Engine ■■■ Model Router ■ ■■■ OpenAI ■ ■■■ Anthropic ■ ■■■ Local Models ■■■ Learning & Memory ■ ■■■ Global ■ ■■■ Project ■ ■■■ Session ■ ■■■ Lessons ■ ■■■ Procedures ■ ■■■ Failure/Recovery Memory ■■■ Project Constitution (OMERTA.md) ■■■ Codebase Intelligence ■ ■■■ Indexer ■ ■■■ Symbol Search ■ ■■■ Dependency Graph ■ ■■■ Git History ■■■ Tool Controller ■ ■■■ Files ■ ■■■ Terminal ■ ■■■ Search ■ ■■■ Git ■ ■■■ Build ■ ■■■ Process Control ■■■ SANDBOX ■ ■■■ Filesystem Isolation ■ ■■■ Process Isolation ■ ■■■ Network Policy ■ ■■■ Resource Limits ■ ■■■ Approval Gates ■ ■■■ Snapshots/Rollback ■■■ Engineering Engines ■ ■■■ App ■ ■■■ Software ■ ■■■ Firmware ■■■ Multi-Agent Team ■ ■■■ Architect ■ ■■■ Developer ■ ■■■ Researcher ■ ■■■ Reviewer ■ ■■■ Tester ■ ■■■ Debugger ■ ■■■ Builder ■ ■■■ Firmware Engineer ■ ■■■ Documentation ■■■ Release / Packaging / Diagnostics

3. Core Features

- Direct/no-bullshit mode: problem → cause → fix → verification.
- Model-agnostic provider layer with routing by task, cost, context and availability.
- Persistent SQLite memory with explicit teach/forget/show/search operations.
- Project Constitution (OMERTA.md) defining hard project rules and priorities.
- Repository indexing and cross-file codebase understanding.
- Search, symbol navigation, dependency/call relationships and Git-aware context.
- Safe tool execution with permission checks and command approval.
- Sandbox-first execution for generated code and engineering commands.

- Build/test/debug loop with automatic evidence collection.
- Failure memory and recovery engine to avoid repeating ineffective fixes.
- Git branches, diffs, commits, rollback, bisect and regression tracking.
- App/software/firmware specialist workflows.
- Android/AOSP/kernel/device-tree/boot-image analysis.
- Hardware knowledge base for datasheets, logs, schematics and register maps.
- Structured artifacts, logs, reports and reproducible build metadata.
- CLI, local web interface and API.
- Security controls for secrets, host access and destructive operations.
- Release packaging for .deb, Appliance and containers.

4. Phase 1 — Installation

Install the base development environment, create the isolated Python environment, install OMERTA dependencies and verify prerequisites.

```
sudo apt update sudo apt install -y git curl wget build-essential python3 python3-venv python3-pip sqlite3
ripgrep jq unzip zip pkg-config ca-certificates # Optional sandbox/build tooling sudo apt install -y podman
bubblewrap git clone cd omerta-ai python3 -m venv .venv source .venv/bin/activate python -m pip install -U pip
pip install -e .
```

Verification: Python, Git, SQLite, search tooling and sandbox capabilities must be detected by `omerta doctor`.

5. Phase 2 — Configuration

```
~/config/omerta/ ███ config.toml ███ memory.db ███ providers.toml ███ policies.toml ███ logs/ Project: ███
OMERTA.md ███ .omerta/ ███ index.db ███ artifacts/ ███ snapshots/ ███ reports/
```

Provider credentials belong in environment variables or a secure credential mechanism. They must never be written into prompts, Git, logs or generated source.

6. Phase 3 — CLI

```
omerta doctor omerta chat omerta agent "task" omerta inspect omerta index omerta search "symbol" omerta build
omerta test omerta debug omerta review omerta teach --scope project "rule" omerta memory list omerta memory
search "term" omerta constitution init omerta sandbox status omerta git status omerta firmware inspect image.dtb
```

7. Phase 4 — Model Providers

Provider abstraction keeps OMERTA independent of a single model vendor. The router selects a backend based on task requirements. OpenAI Responses API, Anthropic Messages API and compatible local inference backends are supported through separate adapters.

Routing categories: planning, coding, large-context analysis, debugging, code review, documentation, local/private tasks and low-cost utility tasks.

8. Phase 5 — Learning & Memory

Memory types: GLOBAL – user-wide engineering preferences PROJECT – project rules and architecture SESSION – current task context FACT – confirmed technical fact LESSON – learned engineering rule PROCEDURE – repeatable workflow FAILURE – failed attempt and evidence SUCCESS – verified successful fix

Every memory item stores scope, source/provenance, timestamp, confidence and status. Explicit user teaching is authoritative unless it conflicts with safety or higher-priority system constraints.

9. Phase 6 — Codebase Intelligence

Index Git repositories into SQLite. Use ripgrep for fast text search and optional language parsers/tree-sitter integrations for symbols. Track files, symbols, imports/includes, definitions, references, tests and build files.

Core commands: `omerta index`, `omerta search`, `omerta symbol`, `omerta graph`, `omerta inspect`.

10. Phase 7 — Tool System

Tools are typed operations rather than unrestricted shell text. Each tool declares required permissions, input validation, sandbox requirements, timeout and whether user approval is required.

Tool groups: filesystem, terminal, process, Git, search, build, test, package, firmware-analysis and device-I/O.

11. Phase 8 — Sandbox

Sandbox is a first-class/default execution layer. Generated code, builds and tests should execute there whenever practical.

Sandbox policy: - Non-root user - Project workspace mounted explicitly - Network disabled by default - CPU limit - Memory limit - PID/process limit - Execution timeout - Read-only base filesystem where practical - Writable workspace only - No host /dev access by default - No host credentials - No SSH keys - No arbitrary host mounts - Snapshot before risky modifications - Rollback on failed workflows when configured

Docker/Podman are preferred isolation backends; bubblewrap can provide lightweight Linux isolation. OMERTA must report when a backend is unavailable rather than falsely claiming isolation.

Host operations, privileged commands, flashing, partition writes and other destructive actions require explicit authorization.

12. Phase 9 — Git Engine

Capabilities: branch creation, clean working tree checks, diff generation, commit generation, patch application, rollback, stash/snapshot, bisect assistance, regression identification and review reports.

Rule: never overwrite uncommitted user work without explicit authorization.

13. Phase 10 — Multi-Agent Orchestration

CENTRAL ORCHESTRATOR ■ ■■■ Architect → architecture / plans ■■■ Researcher → documentation / source analysis ■■■ Developer → implementation ■■■ Reviewer → correctness / maintainability ■■■ Tester → tests / reproduction ■■■ Debugger → root-cause investigation ■■■ Builder → compilation / packaging ■■■ Firmware → kernel / DTS / Android ■■■ Documentation → technical docs

Agents share controlled context and artifacts. The orchestrator resolves conflicting suggestions against the project constitution and evidence.

14. Phase 11 — Build / Test / Debug

Standard loop:

INSPECT → PLAN → MODIFY → BUILD → TEST → DEBUG/FIX → REBUILD → REVIEW → VERIFIED RESULT

Build/test execution records command, environment, exit status, stdout/stderr, duration, artifact paths and hashes. A green model response is not treated as evidence.

15. Phase 12 — Evidence & Recovery

CONFIRMED – directly observed/verified LIKELY – strong interpretation INFERRED – model/tool inference UNKNOWN – insufficient evidence

Recovery memory stores: attempted fix, affected files, exact error, evidence, result, rollback state and successful alternative. OMERTA should use this history to avoid repeating known failures.

16. Phase 13 — App / Software / Firmware Engineering

Domain	Capabilities
App development	Android, desktop, web, CLI, APIs, packaging, tests
Software	Python, C/C++, Rust, Go, Java/Kotlin, shell, build systems
Firmware	ARM/ARM64, bootloaders, kernels, BSP, HAL, drivers, DTS/DTSI/DTB/DTBO
Android	AOSP, Soong/Ninja, vendor partitions, boot images, AVB, OTA
Hardware	Datasheets, schematics, registers, buses, GPIO, power, clocks, logs
Debugging	Crash analysis, logs, compiler errors, kernel panics, boot failures

17. Firmware / Device-Tree Tooling

```
repo / git adb / fastboot dtc / fdt dump / fdtget / fdtput mkbootimg / unpack_bootimg mktbimg avbtool simg2img /  
img2simg lpmake / lpunpack erofs-utils e2fsprogs lz4 / gzip / xz clang / LLVM GCC / cross-compilers ninja / make  
/ Soong
```

OMERTA should inspect device trees and firmware images first, then generate patches or build artifacts. Exact board-level values are extracted from the target hardware/source rather than guessed.

18. Phase 14 — Security & Packaging

Security: - Sandbox by default - Least privilege - No root by default - Explicit approval for destructive actions
- Secret redaction - Safe command parser/policy - Audit logs - Dependency pinning - Artifact hashes - Git safety checks - Network deny-by-default in sandbox

Packaging targets: Debian `.deb`, ApplImage for portable desktop use, Docker/OCI image for isolated server/CI deployments, and source archive.

19. Phase 15 — Release Candidate Stress Test

Test categories: 1. Clean installation 2. Upgrade from previous release 3. Provider failure / timeout 4. Offline mode 5. Invalid model credentials 6. Large repository indexing 7. Concurrent agent tasks 8. Sandbox escape resistance 9. Resource exhaustion 10. Build timeout 11. Compiler failure recovery 12. Git dirty-tree protection 13. Memory persistence 14. Constitution conflict handling 15. Firmware artifact validation 16. Crash/restart recovery 17. Package install/uninstall 18. Reproducibility 19. Secret leakage tests 20. Full end-to-end project build

20. Project Constitution — OMERTA.md

```
# OMERTA AI Project Constitution ## Mission Build reliable software, applications and firmware through
evidence-backed engineering. ## Default behavior Direct. Technical. Concise. No filler. ## Engineering Inspect
before modifying. Build after meaningful changes. Test before claiming success. Record failures. Never invent
missing hardware or source information. ## Sandbox Execute builds/tests in the sandbox by default. ## Git
Protect user changes. Use branches/snapshots for risky work. ## Learning Only explicit teachings and verified
project facts become durable project rules. ## Firmware Never flash hardware automatically. Require explicit
authorization for physical-device writes. ## Evidence No evidence = UNKNOWN.
```

21. Recommended Repository Layout

```
omerta-ai/ ■■■ omerta/ ■ ■■■ core/ ■ ■■■ models/ ■ ■■■ memory/ ■ ■■■ constitution/ ■ ■■■ codebase/ ■ ■■■
sandbox/ ■ ■■■ tools/ ■ ■■■ git/ ■ ■■■ build/ ■ ■■■ debug/ ■ ■■■ firmware/ ■ ■■■ agents/ ■ ■■■ cli/ ■■■
tests/ ■ ■■■ unit/ ■ ■■■ integration/ ■ ■■■ sandbox/ ■ ■■■ security/ ■ ■■■ e2e/ ■■■ web/ ■■■ scripts/ ■■■
packaging/ ■ ■■■ deb/ ■ ■■■ appimage/ ■ ■■■ docker/ ■■■ docs/ ■■■ examples/ ■■■ OMERTA.md ■■■
pyproject.toml ■■■ README.md
```

22. Initial Build / Debug Commands

```
unzip omerta-ai-*.zip cd omerta-ai ./scripts/install.sh source .venv/bin/activate omerta doctor
./scripts/test.sh omerta constitution init omerta sandbox status omerta index omerta agent "Inspect this project
and explain its architecture" omerta review omerta debug "PASTE ERROR HERE" # Build a release artifact
./scripts/build.sh
```

The baseline test is Phase 1 of the debugging cycle. Fixes should be made incrementally, with each phase producing evidence before proceeding.

23. Definition of Done

- Installs cleanly on supported Linux Mint systems.
- Doctor command detects all required/optional dependencies.
- CLI and web/API surfaces operate correctly.
- At least one model provider works end-to-end.
- Memory persists and respects scope.
- OMERTA.md rules are loaded and enforced.
- Repository indexing/search works on real projects.
- Tool permissions prevent unauthorized execution.
- Sandbox executes builds/tests with declared isolation.
- Git safety and rollback work.
- Agent roles can collaborate through the orchestrator.
- Build/test/debug loop records evidence.

- Recovery engine stores and retrieves failed fixes.
- Firmware engine can inspect supported Android artifacts.
- Security tests pass.
- Deb/OpImage/container packaging succeeds.
- 15-phase stress test passes with no unresolved release-blocking defects.

24. Important Implementation Boundary

This PDF is the engineering specification/blueprint. It is not a claim that every Phase 1–15 component already exists in the earlier v0.1 archive. The v0.1 archive is the baseline foundation; the complete implementation is the next engineering build. Features should only be marked COMPLETE after implementation and tests demonstrate them.